S-VYASA
(Deemed to be University u/s 3 of the UGC Act, 1956)
SCHOOL of ADVANCED STUDIES

Swami Vivekananda Yoga Anusandhana Samsthana
स्वामी विवेकानन्द योग अनुसंधान संस्थान
DEEMED TO BE UNIVERSITY

COMPUTER NETWORKS

Prof. Ch. Sarvani
TEAM NETWORKS

Approved by AICTE Ministry of HRD, Govt. of India
A+ NAAC

Module 2

Introduction to Application Layer Protocols, HTTP, SMTP, DNS, Video Streaming and Content Distribution Networks, Socket Programming.

Module 2
Application Layer Protocols

Introduction

The **Application Layer** is the topmost layer in both the **OSI model (Layer 7)** and the **TCP/IP model**. It provides network services directly to end users and applications.

Application layer protocols define the rules for communication between software applications over a network.

Functions of Application Layer

•Provides user interface to network services

•Identifies communication partners

•Synchronizes communication

•Provides services like file transfer, email, web access

•Handles network resource sharing

Major Application Layer Protocols

1 HTTP (HyperText Transfer Protocol)

Purpose:

Used for transferring web pages on the Internet.

Working:

•Client sends HTTP request

•Server sends HTTP response

•Stateless protocol

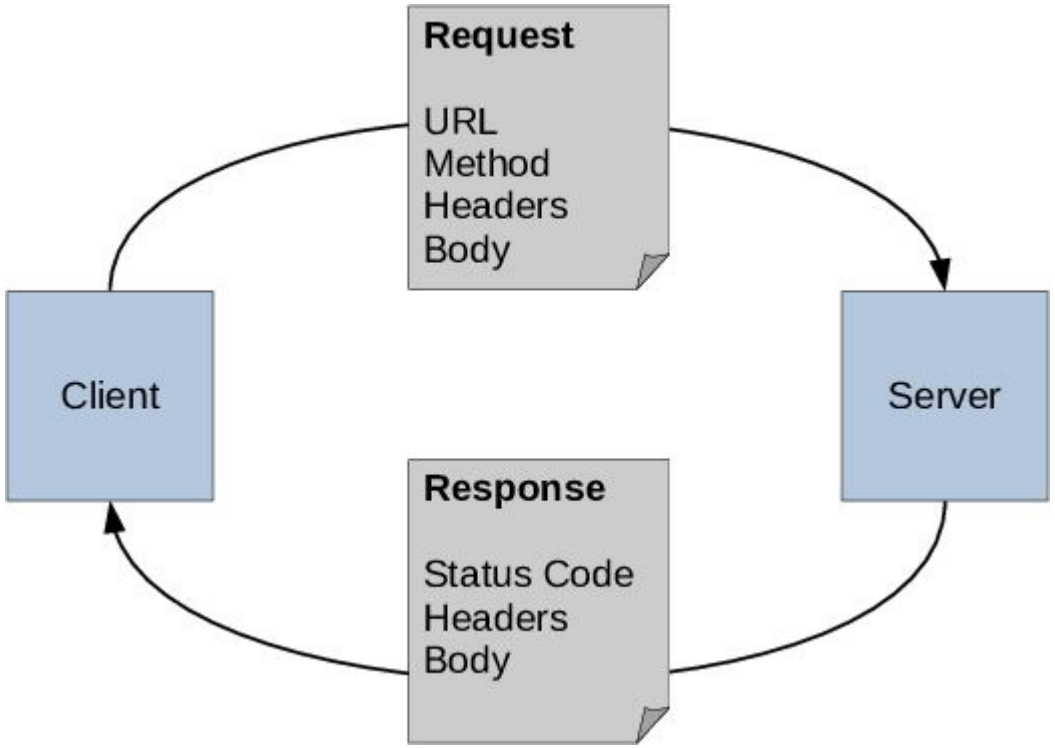
Port Number:

•80 (HTTP)

Features:

•Request/Response model

•Used for web browsing



2 HTTPS (HyperText Transfer Protocol Secure)

Purpose:

Secure version of HTTP.

Features:

•Uses SSL/TLS encryption

•Ensures data confidentiality

•Prevents data tampering

Port Number:

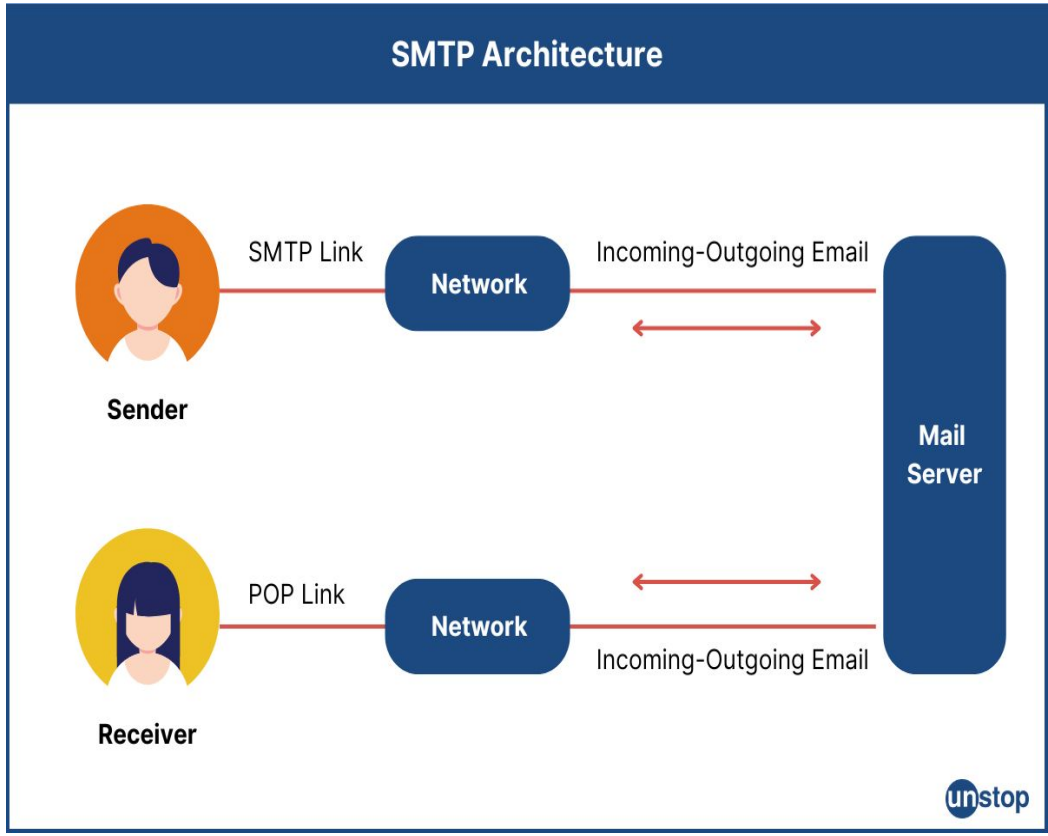
•443

Used in online banking and e-commerce.

Simple Mail Transfer Protocol (SMTP)

- **SMTP (Simple Mail Transfer Protocol)** is an **application layer protocol** used for sending emails over the Internet. It defines how email messages are transferred from the sender's mail client to the mail server and between mail servers.
- SMTP works with other protocols such as **POP3** and **IMAP**, which are used for receiving emails.

- SMTP is a standard protocol used for sending electronic mail messages between mail servers using the TCP/IP



SMTP follows a **client-server architecture** where:

- **Mail User Agent (MUA)** → Email client (e.g., Outlook, Gmail interface)
- **Mail Transfer Agent (MTA)** → Mail server responsible for transferring emails
- **Mail Delivery Agent (MDA)** → Delivers email to recipient mailbox

Step-by-Step Process

1. User composes an email using a mail client.
2. The email is sent to the SMTP server.
3. SMTP server checks the recipient address.
4. The message is transferred to the destination mail server.
5. The recipient retrieves the message using POP3 or IMAP.

Example Commands

Command	Purpose
HELO / EHLO	Initiates connection
MAIL FROM	Specifies sender address
RCPT TO	Specifies recipient address
DATA	Sends message content
QUIT	Terminates session

Example sequence:

HELO mail.example.com
MAIL FROM:<sender@example.com>
RCPT TO:<receiver@example.com>
DATA
Message content
.
QUIT

SMTP Ports

Port	Usage
25	Default SMTP port
587	Secure mail submission
465	SMTP over SSL

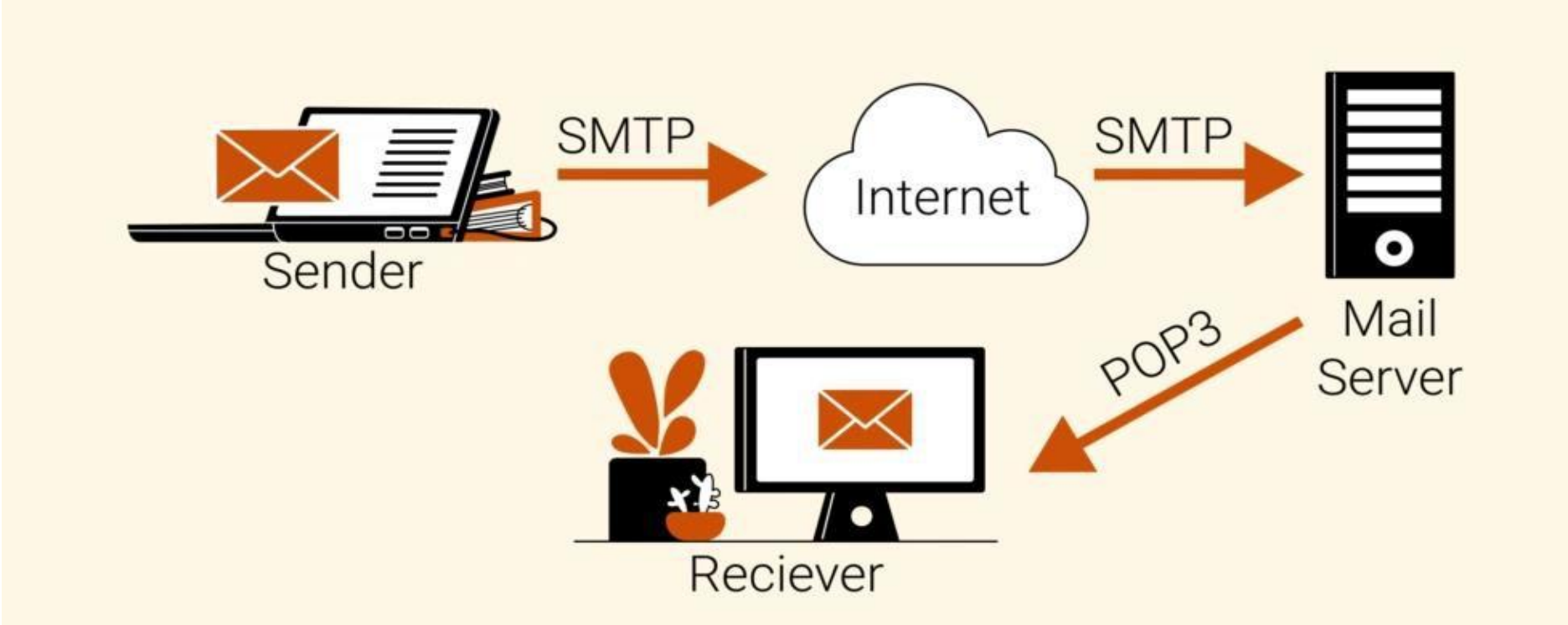
Features of SMTP

- Simple and reliable protocol
- Works on **TCP connection**
- Supports email forwarding between servers
- Uses command-response communication

Post Office Protocol Version 3 (POP3)

- **POP3 (Post Office Protocol Version 3)** is an **application layer protocol** used to retrieve emails from a mail server to a local computer. It allows users to download messages from their mailbox on the mail server and read them offline.

- POP3 works with **SMTP**, which is used to send emails.
- POP3 is a standard application layer protocol used by email clients to retrieve messages from a remote mail server over a TCP/IP network.



POP3 uses a **client-server architecture**.

Components

- **Mail User Agent (MUA)** – Email client (Outlook, Thunderbird, etc.)
- **Mail Server** – Stores emails until retrieved
- **POP3 Server** – Handles email retrieval requests

Working of POP3

Step-by-Step Process

1. User connects to mail server using POP3.
2. Server authenticates username and password.
3. Client downloads messages from mailbox.
4. Emails are stored locally on the user's device.
5. Messages are usually deleted from server after download.

POP3 Communication Phases

POP3 operates in **three phases**:

1 Authorization Phase

- Client sends username and password.
- Server verifies the user.

Example commands:
USER
PASS

2 Transaction Phase

- Client retrieves emails.
- Messages can be marked for deletion.

Commands used:

LIST – Lists messages

RETR – Retrieve message

DELE – Delete message

3 Update Phase

- Server deletes messages marked for deletion.
- Connection is

terminated. Command

used:

QUIT

POP3 Port Numbers

Port	Usage
110	Default POP3 port
995	Secure POP3 (POP3S)

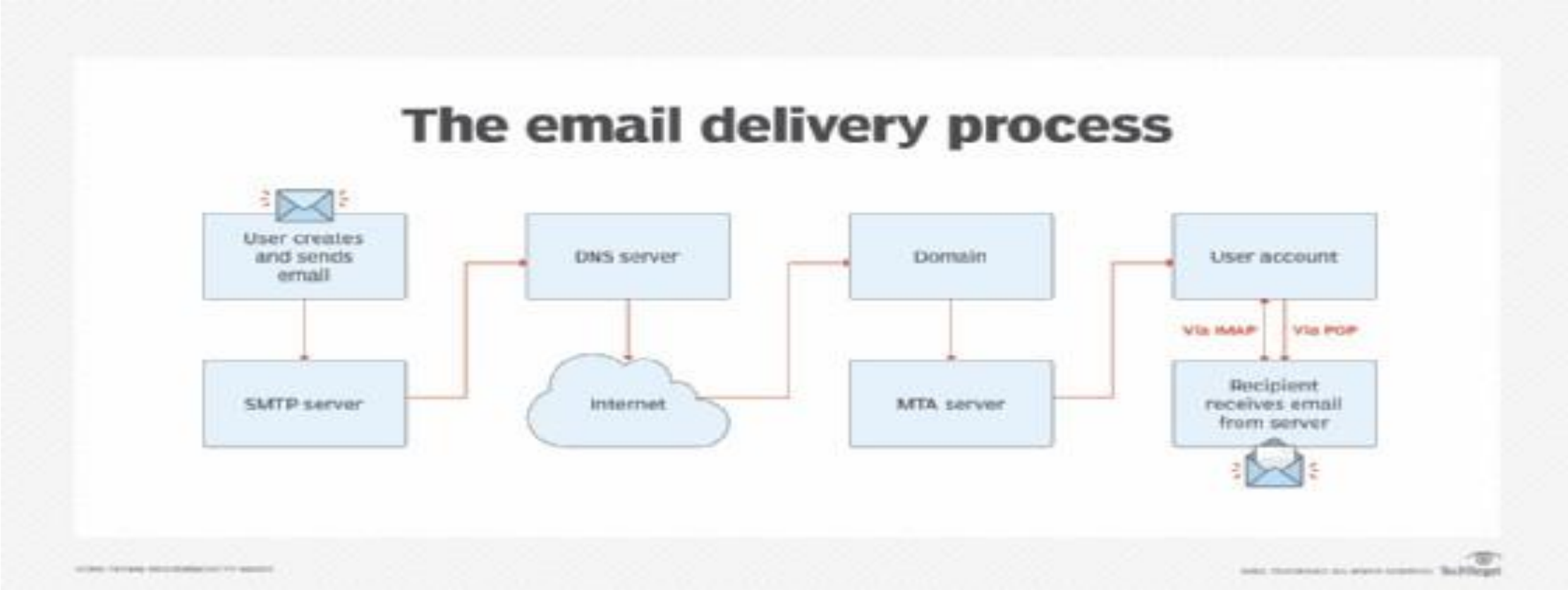
Features of POP3

- Simple email retrieval protocol
- Allows offline email reading
- Downloads emails to local machine
- Reduces server storage usage

Internet Message Access Protocol (IMAP)

IMAP (Internet Message Access Protocol) is an **application layer protocol** used to access and manage emails stored on a mail server. Unlike POP3, IMAP allows users to read emails directly from the server without downloading them permanently to the local device.

IMAP is widely used in modern email systems because it supports **synchronization across multiple devices**.
IMAP is a standard application layer protocol that allows users to access, manage, and organize email messages stored on a mail server over a TCP/IP network.


The email delivery process
User creates and sends email
SMTP server
DNS server
Internet
Domain
MTA server
User account
Via IMAP
Via POP
Recipient receives email from server

IMAP follows a **client-server architecture**.

Components

- **Mail User Agent (MUA)** – Email client (Gmail, Outlook, Thunderbird)
- **IMAP Server** – Stores and manages emails
- **Mail Server** – Maintains user mailbox

Working of IMAP

Step-by-Step Process

1. User logs in to the email client.
2. Client connects to the IMAP server.
3. Server authenticates the user.
4. Emails remain stored on the server.
5. User can read, delete, or organize emails remotely.
6. Changes are synchronized across all devices.

IMAP Operations

IMAP allows many mailbox management operations.

Common IMAP Commands

Command	Function
LOGIN	Authenticate user
SELECT	Select mailbox
FETCH	Retrieve email
STORE	Modify message flags
SEARCH	Search messages
LOGOUT	Close connection

IMAP Communication Phases

IMAP works through several operations such as:

1. **Connection Establishment** – Client connects to server
2. **Authentication** – Username and password verification
3. **Mailbox Selection** – Choose inbox or folder
4. **Message Retrieval** – Access or download emails
5. **Session Termination** – Close connection

IMAP Port Numbers

Port	Usage
143	Default IMAP port
993	Secure IMAP (IMAPS)

Features of IMAP

- Emails stored on server
- Supports multi-device synchronization
- Allows folder management
- Supports partial email download
- Enables email searching on server

Advantages of IMAP

- Access email from multiple devices
- Real-time synchronization
- Better email management
- Server-based storage

Disadvantages of IMAP

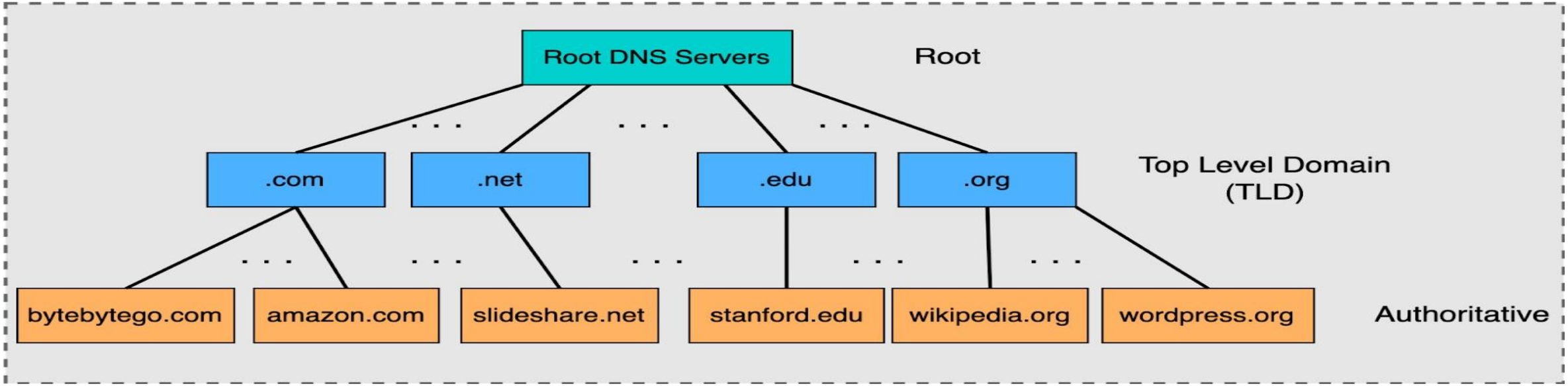
- Requires continuous internet connection
- Uses more server storage
- Slightly slower than POP3

Domain Name System (DNS)

The **Domain Name System (DNS)** is an application layer protocol used to translate human-readable domain names into machine-readable IP addresses. It acts like a directory service for the Internet.

For example, when a user types **www.google.com**, DNS converts the domain name into an IP address such as **142.250.190.78**, allowing computers to locate the correct server.

DNS is a distributed naming system that translates domain names into IP addresses so that computers can communicate with each other over a network.



DNS follows a **hierarchical and distributed architecture** consisting of several types of servers.

Components of DNS

- **DNS Resolver (Client)**
- Initiates DNS query from user's computer.
- **Root Name Server**
- Top-level server in DNS hierarchy.
- Directs queries to appropriate TLD servers.
- **Top-Level Domain (TLD) Server**
- Manages domain extensions such as .com, .org, .edu.
- **Authoritative Name Server**
- Stores actual IP address for a domain.

Working of DNS

Step-by-Step Process

1. User enters a domain name in the browser.
2. The browser sends a DNS request to a DNS resolver.
3. Resolver contacts the **Root Server**.
4. Root server directs the request to the **TLD server**.
5. TLD server directs it to the **Authoritative server**.
6. Authoritative server returns the IP address.
7. Browser connects to the server using that IP address.

DNS Query Types

1 Recursive Query

- DNS resolver performs the complete lookup and returns the final IP address.

2 Iterative Query

- DNS server returns the best possible answer and refers the client to another server.

DNS Record Types

Record Type	Purpose
A	Maps domain to IPv4 address
AAAA	Maps domain to IPv6 address
CNAME	Alias for another domain
MX	Mail server record
NS	Name server record
PTR	Reverse DNS lookup

DNS Port Number

- **Port 53**
- Uses both:
  - **UDP** for normal queries
  - **TCP** for large responses and zone transfers

Features of DNS

- Distributed database system
- Hierarchical naming structure
- Fast name resolution
- Supports domain aliases
- Scalable for global Internet use

Advantages of DNS

- Easy-to-remember domain names
- Efficient internet navigation
- Reduces need to remember IP addresses
- Supports large-scale network communication

Video Streaming and Content Distribution Networks (CDN)

1 Video Streaming

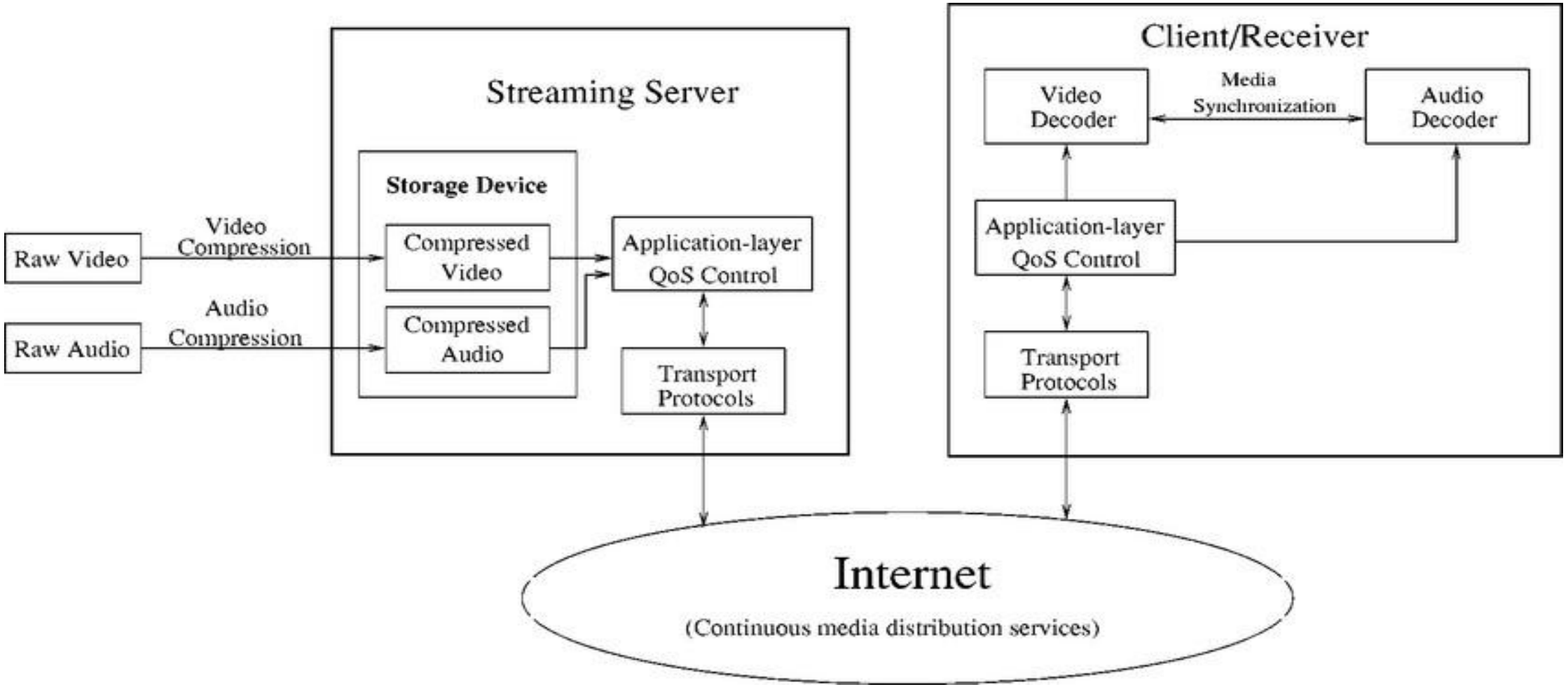
Introduction

Video streaming is a technology that allows users to watch video content over the Internet without downloading the entire file first. The video data is transmitted in small packets and played continuously as it arrives.

Streaming enables real-time viewing of videos on platforms such as **YouTube**, **Netflix**, and **Amazon Prime Video**.

Step-by-Step Process

1. User requests a video from a streaming server.
2. Server divides the video into small segments.
3. Segments are transmitted over the network.
4. Media player buffers and decodes the video.
5. Video plays continuously while remaining data downloads.


Streaming Server
Storage Device
Raw Video
Video Compression
Compressed Video
Raw Audio
Audio Compression
Compressed Audio
Application-layer QoS Control
Transport Protocols
Client/Receiver
Video Decoder
Media Synchronization
Audio Decoder
Application-layer QoS Control
Transport Protocols
Internet
(Continuous media distribution services)

Types of Video Streaming

1 Live Streaming

- Video broadcast in real-time.
- Example: sports events, online classes.

2 On-Demand Streaming

- Pre-recorded videos available anytime.
- Example: movies on streaming platforms.

Major Streaming Protocols

1 RTSP (Real Time Streaming Protocol)

Definition

RTSP is a network control protocol used for controlling streaming media servers.

Features

- Provides control commands like **play, pause, stop**
- Works with RTP for media transmission
- Used in IP cameras and media streaming systems

Common Commands

- PLAY
- PAUSE
- SETUP
- TEARDOWN

Port

- Default port: **554**

2 RTP (Real-time Transport Protocol)

Definition

RTP is used to deliver real-time audio and video data over IP networks.

Features

- Supports **real-time communication**
- Provides **timestamp and sequence numbers**
- Works with **RTSP** for quality monitoring

Applications

- Video conferencing
- Live streaming
- VoIP calls

3 RTMP (Real Time Messaging Protocol)

Definition

RTMP is a protocol developed by **Adobe** for high-performance streaming of audio, video, and data.

Features

•Low-latency streaming

•Persistent TCP connection

•Used for live broadcasting

Usage

•Live streaming platforms

•Video broadcasting services

4 HLS (HTTP Live Streaming)

Definition

HLS is a streaming protocol developed by **Apple** for delivering video over HTTP.

Features

•Breaks video into small segments

•Supports adaptive bitrate streaming

•Works over standard HTTP

Advantages

•Compatible with firewalls

•Widely supported on mobile devices

5 MPEG-DASH (Dynamic Adaptive Streaming over HTTP)

Definition

MPEG-DASH is an adaptive streaming protocol that adjusts video quality based on network conditions.

Features

•Dynamic bitrate adaptation

•Works over HTTP

•Standardized by MPEG

Applications

•Video-on-demand platforms

•Online video streaming services

2 Content Distribution Network (CDN)

Introduction

A **Content Distribution Network (CDN)** is a network of geographically distributed servers that deliver web content to users based on their location. It improves performance by delivering data from the nearest server.

Components of CDN

1.Origin Server

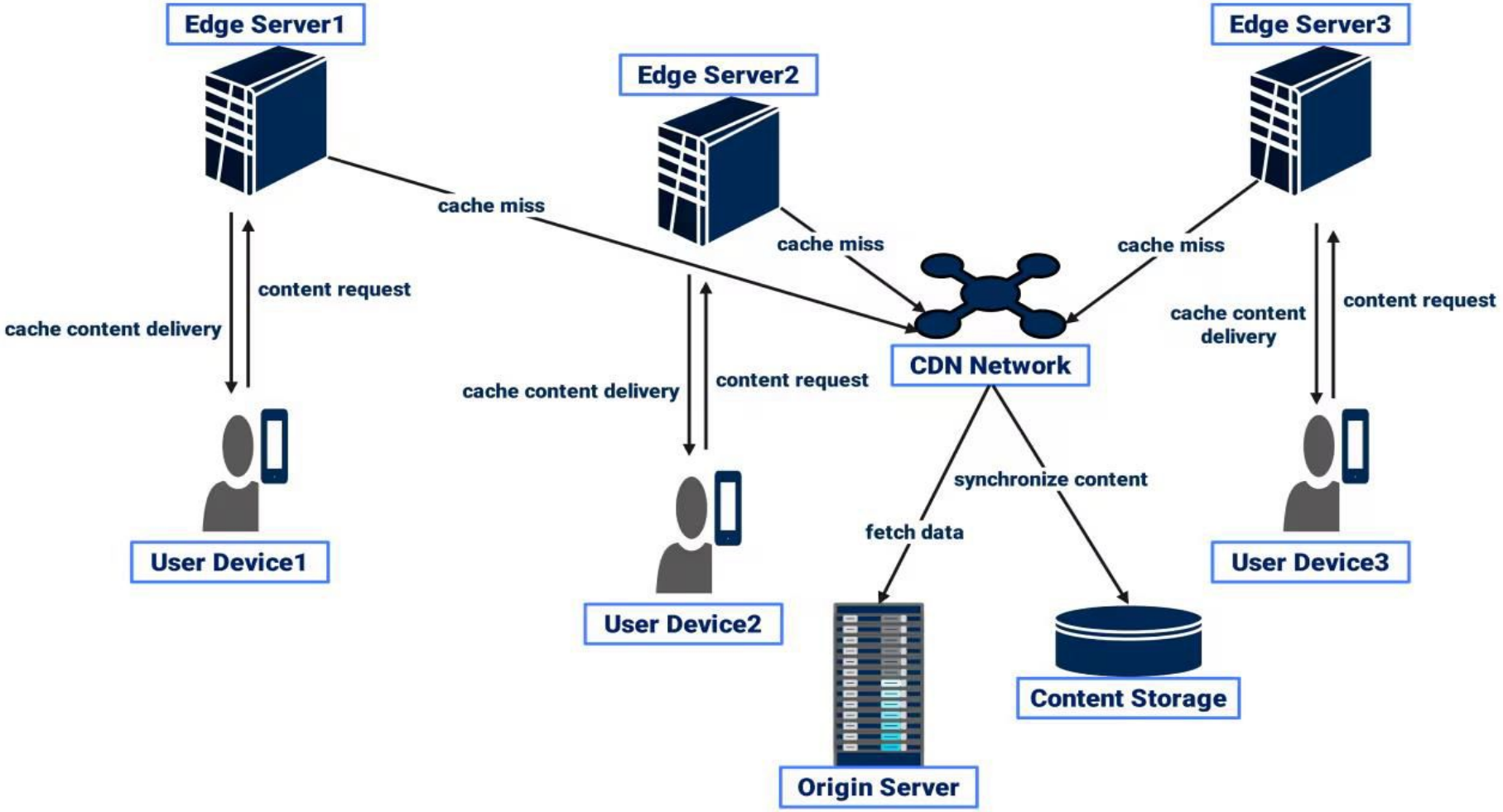
1. Main server storing original content.

2. Edge Servers

Distributed servers closer to users.

3. Caching System

Stores copies of frequently accessed content.


Edge Server1
Edge Server2
Edge Server3
cache miss
cache miss
cache miss
content request
cache content delivery
cache content delivery
content request
cache content delivery
content request
CDN Network
synchronize content
fetch data
User Device1
User Device2
User Device3
Content Storage
Origin Server

Working of CDN

1. User requests content from a website.
2. DNS redirects request to nearest CDN server.
3. Edge server delivers cached content.
4. If content is unavailable, request is forwarded to origin server.
5. Content is delivered quickly to the user.

Benefits of CDN

- Faster content delivery
- Reduced latency
- Lower server load
- Improved website performance
- Better user experience

Applications of CDN

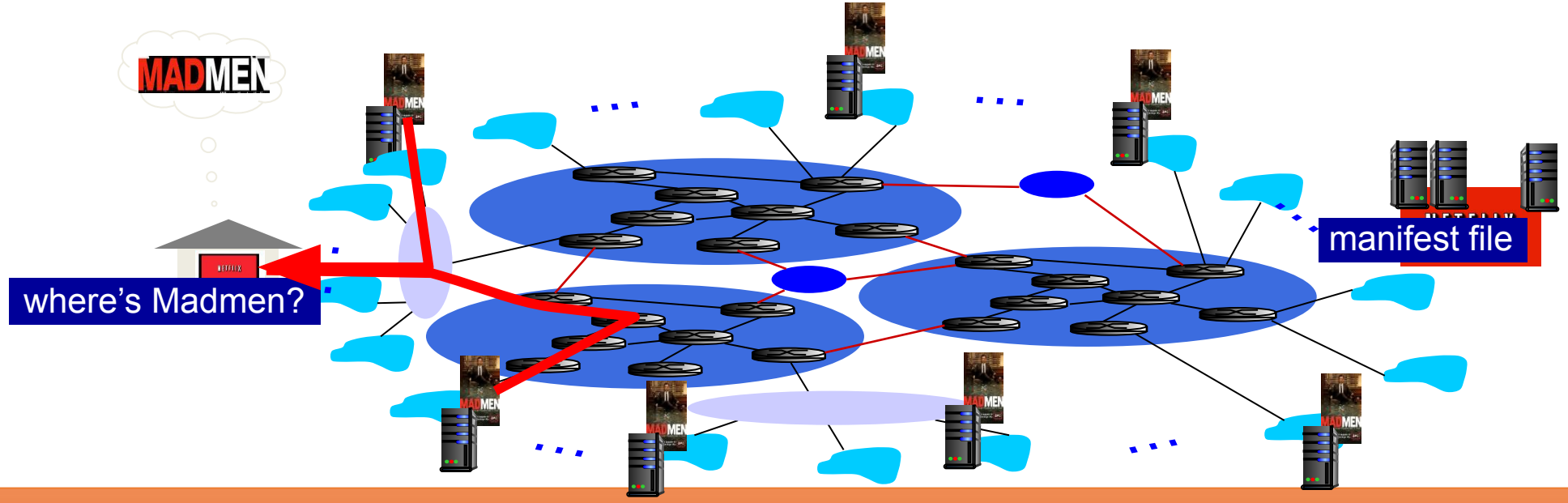
- Video streaming platforms
- Online gaming services
- E-commerce websites
- Social media platforms
- Software distribution

Relationship Between Video Streaming and CDN

CDNs are widely used to deliver streaming content efficiently. Instead of sending video from a single server, CDNs distribute video segments across multiple servers worldwide, reducing buffering and improving streaming quality.

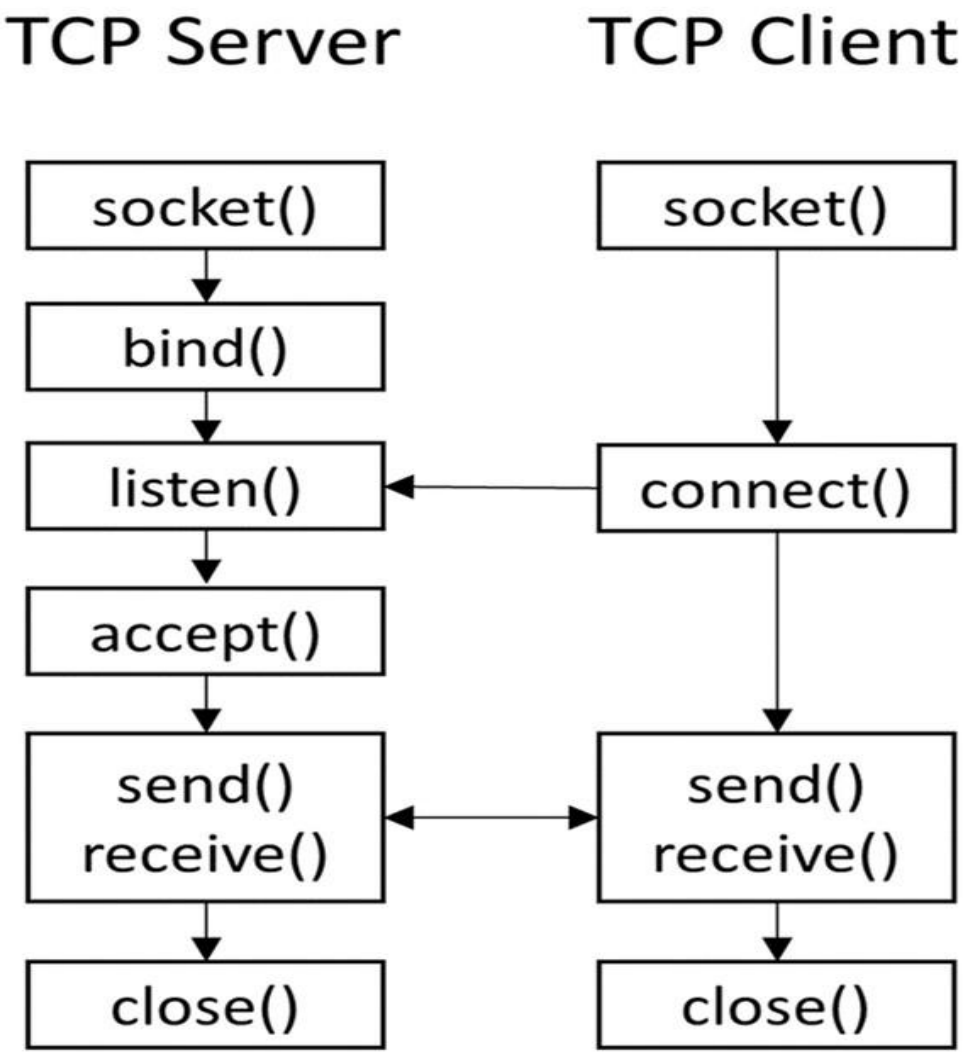
How does Netflix work?

- Netflix: stores copies of content (e.g., MADMEN) at its (worldwide) OpenConnect CDN nodes
- subscriber requests content, service provider returns manifest
  - using manifest, client retrieves content at highest supportable rate
  - may choose different rate or copy if network path congested



Socket Programming in Computer Networks

- **Socket programming** is a method used for communication between two computers over a network using sockets. A socket is an endpoint for sending or receiving data across a network.
- It allows client and server programs to communicate with each other using protocols such as TCP or UDP.
- Socket programming is widely used in applications such as chat applications, web servers, file transfer systems, and online games.
- A socket is an endpoint of communication between two processes running on different machines connected through a network.


TCP Server
TCP Client
socket()
bind()
listen()
accept()
send()
receive()
close()
socket()
connect()
send()
receive()
close()

Socket communication generally follows the **client-server model**.

Components

1. **Client** – Requests services from the server.
2. **Server** – Provides services to the client.
3. **Socket** – Communication endpoint between client and server.
4. **Port Number** – Identifies the application process.

Types of Sockets

1 Stream Sockets (TCP)

- Uses **Transmission Control Protocol (TCP)**
- Connection-oriented communication
- Reliable data transfer
- Used in web browsing and email systems

2 Datagram Sockets (UDP)

- Uses **User Datagram Protocol (UDP)**
- Connectionless communication
- Faster but less reliable
- Used in streaming and online gaming

Socket Programming Process

Server Side Steps

1.Create socket

2.Bind socket to port number

3.Listen for connection

4.Accept client request

5.Send/receive data

6.Close connection

Client Side Steps

1. Create socket
2. Connect to server
3. Send request
4. Receive response
5. Close connection

Socket Functions with Syntax and Arguments

In **socket programming**, different socket functions are used to establish communication between client and server applications over a network. These functions are provided by the **Socket API** and are mainly used in programming languages like C, C++, Java, and Python.

These functions allow programs to create sockets, establish connections, transmit data, and close communication.

Important Socket Functions

The most commonly used socket functions are:

socket()

bind()

listen()

accept()

connect()

send()

recv()

close()

1 socket() Function

Purpose

Creates a new socket and returns a socket descriptor.

Syntax

int socket(int domain, int type, int protocol);

Argument	Description
domain	Specifies the communication domain such as IPv4 or IPv6
type	Specifies the communication type
protocol	Specifies the protocol used

Common Values

Parameter	Value	Meaning
domain	AF_INET	IPv4 protocol
type	SOCK_STREAM	TCP connection
type	SOCK_DGRAM	UDP connection
protocol	0	Default protocol

2 bind() Function

Purpose

Associates the socket with a specific IP address and port number.

Syntax

int bind(int sockfd, struct sockaddr *addr, socklen_t addrlen);

Argument	Description
sockfd	Socket descriptor returned by socket()
addr	Pointer to socket address structure
addrlen	Size of the address structure

3 listen() Function

Purpose

Places the server socket in passive mode to listen for incoming client requests.

Syntax

int listen(int sockfd, int backlog);

Argument	Description
sockfd	Socket descriptor
backlog	Maximum number of queued connection requests

4 accept() Function

Purpose

Accepts an incoming client connection request.

Syntax

int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);

Argument	Description
sockfd	Server socket descriptor
addr	Address structure for client
addrlen	Length of client address

5 connect() Function

Purpose

Establishes a connection between client and server.

Syntax

int connect(int sockfd, struct sockaddr *addr, socklen_t addrlen);

Argument	Description
sockfd	Client socket descriptor
addr	Server address structure
addrlen	Size of address structure

6 send() Function

Purpose

Sends data from one socket to another.

Syntax

send(int sockfd, const void *buf, size_t len, int flags);

Argument	Description
sockfd	Socket descriptor
buf	Data buffer to send
len	Size of data
flags	Control flags

7 recv() Function

Purpose

Receives data from a socket.

Syntax

recv(int sockfd, void *buf, size_t len, int flags);

Argument	Description
sockfd	Socket descriptor
buf	Buffer to store received data
len	Maximum length of data
flags	Control flags

8 close() Function

Purpose

Closes the socket connection.

Syntax

int close(int sockfd);

Argument	Description
sockfd	Socket descriptor to close

let's use a real-world analogy: **Ordering Pizza over the Phone**

1. The TCP (Connection-Oriented) Example

TCP is like a phone call. You must establish a connection before any information is exchanged.

The Server (Pizza Shop):

Create Socket: The shop installs a phone line.

Bind: They assign a specific phone number (IP Address) and an extension for orders (Port, e.g., Port 80).

Listen: The employee sits by the phone waiting for it to ring.
Accept: The phone rings; the employee picks up. A dedicated "line" is now open between the shop and that specific customer.

- **The Client (You):**
  1. **Create Socket:** You pick up your phone.
  2. **Connect:** You dial the shop's number.
- **Data Exchange:** You say your order; they confirm they heard it. If the line is fuzzy, you ask "What?" and they repeat it (**Reliability/Retransmission**).
- **Close:** Both parties hang up.

Real-World Code Logic (Python Style)

If you were writing this for a **Web Server** (Chapter 2 concept), the logic looks like this:

Server-Side:

1. `serverSocket = socket(AF_INET, SOCK_STREAM)` (Create the phone)
2. `serverSocket.bind(('', 12000))` (Assign the number)
3. `serverSocket.listen(1)` (Wait for a "ring")
4. `connectionSocket, addr = serverSocket.accept()` (Pick up the phone)
5. `data = connectionSocket.recv(1024)` (Listen to the order)

Client-Side:

1. `clientSocket = socket(AF_INET, SOCK_STREAM)` (Pick up your phone)
2. `clientSocket.connect((serverName, 12000))` (Dial the number)
3. `clientSocket.send(message)` (Speak the order)

Summary of Socket Functions

Function	Purpose
socket()	Creates socket
bind()	Assigns IP and port
listen()	Waits for connections
accept()	Accepts client request
connect()	Connects client to server
send()	Sends data
recv()	Receives data
close()	Closes connection

THANK YOU

TEAM NETWORKS

Department of Computer Science and Engineering